

2024 Sept OPPE 1 - SET 2

Section 2 - Problem 2

Problem Statement

Problem statement:

- You are given n pairs of integers. For each pair, you need to alternately calculate:
 - Sum of the pair for the odd-numbered pairs (1st, 3rd, 5th, ...)
 - Absolute difference of the pair for the even-numbered pairs (2nd, 4th, 6th, ...)
- The program should read input from the user and print result for each pair in order.
- **Input Format:**
First line: An integer n , the number of pairs. Next n lines: Each line contains two comma-separated integers.
- **Output Format:** For each pair, print the sum (odd-numbered pair) or absolute difference (even-numbered pair) on a new line.

Example and Explanation:

Example:

Input:

3
1,2
3,5
4,3

Output:

3
2
7

First pair (1, 2) → Odd-numbered → Sum: $1+2 = 3$

Second pair (3, 5) → Even-numbered → Absolute difference: $|3 - 5| = 2$

Third pair (4, 3) → Odd-numbered → Sum: $4 + 3 = 7$

Hence, the final output is: 3 2 7

Logic Visualization of our Problem

Reading the Input

```
input for n; for i in  
range (n)
```

Take the number of pairs n.

Read the next n lines, each containing two comma-separated integers using for loop.

Parse the Pairs into list of tuples

```
string.split(","),  
datatype conversion
```

Split each line by comma and convert the values to integers.

Store each pair as a tuple or list for processing.

Process Pairs Alternately

```
for i in range, if else  
statements
```

Use a loop with an index to track odd or even pairs.

If the pair index is odd
→ calculate $\text{sum}(a, b)$.

If the pair index is even
→ calculate $\text{abs}(a-b)$.

Print the Output

```
list enumeration
```

- Print the result of each pair on a new line.
- Maintain the order of the input.

Concept of Conditional Processing in Loops

- In Python, loops (for or while) can be combined with conditional statements to perform different operations based on specific criteria.
- **For Loop:** Iterates through elements or indices sequentially.
- **While Loop:** Uses a condition and index to control iteration.
- **Conditional Statements (if-else):** Decide which operation to perform during each iteration. For example, calculating sum for odd-numbered pairs and absolute difference for even-numbered pairs.
- **Enumerate:** Provides both the index and value, making it easy to apply conditions based on position.

Example using for loop with enumerate:

```
pairs = [(1, 2), (3, 5), (4, 3)]

for i, (a, b) in enumerate(pairs):

    if idx % 2 == 0:    # Odd-numbered pair

        print(a + b)

    else:                # Even-numbered pair

        print(abs(a - b))
```

Takeaway:

Combining iteration with conditional logic allows Python programs to process elements differently based on position, value, or any other criteria, making loops highly versatile for tasks like alternating operations, filtering, or counting.

Output: 3 2 7

Concept of String Splitting, Type Conversion & Math Operations

String Splitting (split):

Used to separate a string into multiple parts based on a delimiter. Essential for parsing comma-separated input into individual numbers.

Data Type Conversion (int())

Converts strings to integers for arithmetic operations. Without conversion, arithmetic operations on strings would result in concatenation instead of sum.

Math Operations (abs())

`abs(x - y)` computes the absolute difference, ensuring a non-negative result regardless of order.

Example:

```
line = "3,5"

a, b = map(int, line.split(','))    # split + type conversion

sum_val = a + b                      # sum

diff_val = abs(a - b)               # absolute difference

print(sum_val, diff_val)
```

Output: 8 2

Our Solution Approach



The Explanatory Approach

Step-by-step implementation with if else statements for learning and understanding each concept



Alt way: Using enumerate()

Using built in python functional programming technique called enumerate() for index tracking



The Concise One-Liner Approach

Uses the single line List Comprehension Approach

Each solution produces identical results but demonstrates different Python programming techniques.

Solution 1: The Explanatory Approach

```
n = int(input())
l = []
for i in range(n):
    num = input()
    l.append(num.split(","))
for i in range(len(l)):
    a, b = int(l[i][0]), int(l[i][1])
    if i % 2 == 0:
        print(a + b)
    else:
        print(abs(a-b))
```

The most beginner friendly way to solve this problem

Solution 2: Alt way Using Functional programming

```
pairs = [input() for _ in range(int(input()))]
for idx, line in enumerate(pairs):
    a, b = map(int, line.split(', '))
    if idx % 2 == 0:
        print(a + b)
    else:
        print(abs(a - b))
```

How this works:

- **enumerate()** function allows you to easily access both the index and the value of each item simultaneously, which is especially useful when looping. **[(index, element), (1, 'banana'), (2, 'cherry')]**
- **map(function, iterable)** : applies a specific function to every element of iterable, `map(int, line.split(','))` converts the elements in the list made from the split function into integer datatype.

An alternative solution using functional programming

Solution comparison

Approach	Pros	Cons	Best For
Explanatory Approach	Highly readable, easy to debug, best for learning, clear variable names	More lines of code, multiple variables	Educational purposes, beginners, complex debugging
Functional Programming	Balanced readability and conciseness, single return statement, fewer variables	Requires understanding and practice of functions like <code>map()</code> and <code>enumerate()</code>	Production code, intermediate developers,

Key Takeaways

- 🔑 String Splitting (`split`) allows parsing comma-separated input into separate elements.
- 🔑 Data Type Conversion (`int()`) is necessary to convert input strings to integers for arithmetic operations.
- 🔑 Math Operations (`+` and `abs()`) are used to calculate sum and absolute difference of pairs.
- 🔑 Conditional Statements (`if-else`) allow alternating between sum and absolute difference based on the pair's position (odd/even).
- 🔑 Enumerate can be used to get both index and value, making it easier to decide which operation to perform.
- 🔑 Combining iteration, conditionals, type conversion, and math operations allows efficient and accurate processing of input data.